

1. NodeMCU 디지털 입출력 – PB 누르면 LED 켜지기

```
#define LEDS 3
const byte ledpins[LEDS] = {D5, D6, D7};
int btnpin = D4;

void setup() {
    LedInit();
    pinMode(btnpin, INPUT);
    Serial.begin(115200);
}

void loop() {
    Serial.print("Pushbutton is ");
    if (digitalRead(btnpin)) { // 버튼을 누른 상태
        Serial.println("pressed!");
        LedOn();
    } else { // 버튼을 누르지 않은 상태
        Serial.println("unpressed!");
        LedOff();
    }
    delay(1000);
}

void LedOn() {
    for (int i=0; i<LEDS; i++) {
        digitalWrite(ledpins[i], HIGH);
    }
    delay(500);
}

void LedOff() {
    for (int i=0; i<LEDS; i++) {
        digitalWrite(ledpins[i], LOW);
    }
    delay(500);
}
```

2. NodeMCU 아날로그 입출력 – 조도센서(CDS)로 LED 밝기 조절

1) CDS센서를 이용한 아날로그 입력

```
#define LED_PIN D0
#define POT_PIN A0    // CDS 조도센서 → A0 핀

void setup() {
    Serial.begin(115200);
}

void loop() {
    int input = analogRead(POT_PIN);    // 0~1023

    // 300~700 범위의 input을 255~0 (역방향)으로 매핑 후 제한
    int duty = constrain(map(input, 300, 700, 255, 0), 0, 255);

    Serial.print("input : ");
    Serial.print(input);
    Serial.print(" --> duty : ");
    Serial.println(duty);

    analogWrite(D0, duty);    // PWM으로 LED 밝기 조절
    delay(500); // 지연시간
}
```

2) Ticker 인터럽트 이용 CDS센서 입력

```
#define LED_PIN D0 //아날로그 출력핀 (D0)
```

```
#define POT_PIN A0 //아날로그 입력핀 (A0)
```

```
Ticker mTicker; //Ticker 레퍼런스 변수 선언
```

```
int input; //측정값을 저장할 전역변수
```

```
void readCdS(){
```

```
    input = analogRead(POT_PIN);
```

```
}
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    mTicker.attach(1.0, readCdS);
```

```
}
```

```
void loop() {
```

```
    int duty = constrain(map(input, 300, 700, 255, 0), 0, 255);
```

```
    Serial.print("input : ");
```

```
    Serial.print(input);
```

```
    Serial.print(" --> duty : ");
```

```
    Serial.println(duty);
```

```
    analogWrite(LED_PIN, duty);
```

```
    delay(500); // 지연시간
```

```
}
```

3. NodeMCU 무선제어 – WiFiServerLED On/Off

```
#include <ESP8266WiFi.h>

const char* ssid      = "khr";
const char* password = "12345678";

int ledPin = D4;
WiFiServer server(80);  // 서버 모듈(객체) 생성

void setup() {
    Serial.begin(115200);
    delay(10);
    pinMode(ledPin, OUTPUT);
    digitalWrite(ledPin, LOW);

    Serial.println();
    Serial.print("Connecting to ");
    Serial.println(ssid);

    WiFi.begin(ssid, password);    // WiFi 네트워크에 연결

    // WiFi 연결될 때까지 대기
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("");
    Serial.println("WiFi connected");

    server.begin();                // 서버 동작 시작
    Serial.println("Server started");

    // 서버 IP 주소 출력 → 클라이언트에서 접속할 때 사용
    Serial.print("Use this URL to connect: ");
    Serial.print("http://");
    Serial.print(WiFi.localIP());
    Serial.println("/");
}

void loop() {
    // 클라이언트 접속 확인 → 연결되면 클라이언트 정보를 갖고 옴
    WiFiClient client = server.available();
```

```

if (!client) {
    return;
}

// 클라이언트가 데이터를 보낼 때까지 기다림
Serial.println("new client");
while (!client.available()) {
    delay(1);
}

// 클라이언트가 보낸 신호의 첫째 줄을 읽고, 시리얼 모니터로 출력
String request = client.readStringUntil('\r');
Serial.println(request);
client.flush();

// 응답에 따른 동작
int value = LOW;
if (request.indexOf("/LED=ON") != -1) {
    digitalWrite(ledPin, HIGH);
    value = HIGH;
}
if (request.indexOf("/LED=OFF") != -1) {
    digitalWrite(ledPin, LOW);
    value = LOW;
}

// 클라이언트로 보내는 웹 페이지 정보 (HTML)
client.println("HTTP/1.1 200 OK");
client.println("Content-Type: text/html");
client.println(""); // 이 줄이 꼭 필요함
client.println("<!DOCTYPE HTML>");
client.println("<html>");

// 현재 LED 상태를 표시함
client.print("Led is now: ");
if (value == HIGH) {
    client.print("On");
} else {
    client.print("Off");
}

// 클라이언트가 선택할 수 있는 버튼 배치

```

```
client.println("<br><br>");
client.println("<a href=\" /LED=ON\"><button>On </button></a>");
client.println("<a href=\" /LED=OFF\"><button>Off </button></a><br />");
client.println("</html>");

delay(1);
Serial.println("Client disconnected");
Serial.println("");
}
```

4. Raspberry Pi 디지털 입출력 — 토글 스위치로 LED On/Off

1) RPI.GPIO 버전

상승에지를 이용한 toggle button

```
import RPi.GPIO as GPIO
```

```
btnPin = 27
```

```
ledPin = 17
```

```
GPIO.setwarnings(False)
```

```
GPIO.setmode(GPIO.BCM)
```

```
# GPIO.setup(btnPin, GPIO.IN, pull_up_down=GPIO.PUD_DOWN) # 풀다운 (주석처리)
```

```
GPIO.setup(btnPin, GPIO.IN, pull_up_down=GPIO.PUD_UP) # 풀업 사용
```

```
GPIO.setup(ledPin, GPIO.OUT)
```

이전 버튼값과 LED 상태를 표시할 변수 선언

```
preBtnInput = False
```

```
ledOn = False
```

```
try:
```

```
    while True:
```

```
        curBtnInput = GPIO.input(btnPin)
```

```
        # 상승에지(rising edge)에 토글스위치 동작
```

```
        if not curBtnInput and preBtnInput:
```

```
            ledOn = not ledOn
```

```
        # ledOn 상태값에 따라 LED 동작
```

```
        GPIO.output(ledPin, ledOn)
```

```
        # 동일 입력에 대응하지 않도록 이전 입력값을 변경
```

```
        preBtnInput = curBtnInput
```

```
except KeyboardInterrupt:
```

```
    pass
```

```
GPIO.cleanup()
```

2) gpiozero 버전

```
from gpiozero import Button, LED
from time import sleep

preBtnInput = False
ledOn = False

btnPin = 27
ledPin = 17

button = Button(btnPin)
led = LED(ledPin)

try:
    while True:
        curBtnInput = button.is_pressed
        if curBtnInput and not preBtnInput:
            ledOn = not ledOn
            preBtnInput = curBtnInput
            if ledOn:
                led.on()
                sleep(0.1)
            else:
                led.off()
                sleep(0.1)
except KeyboardInterrupt:
    pass
```


3) gpiozero 간결 버전

```
from gpiozero import Button, LED
from time import sleep
```

```
preBtnInput = False
```

```
ledOn = False
```

```
btnPin = 27
```

```
ledPin = 17
```

```
button = Button(btnPin)
```

```
led = LED(ledPin)
```

```
try:
```

```
    while True:
```

```
        curBtnInput = button.is_pressed
```

```
        if curBtnInput and not preBtnInput:
```

```
            led.toggle()    # toggle() 메서드로 한 줄로 처리!
```

```
            sleep(0.1)
```

```
            preBtnInput = curBtnInput
```

```
except KeyboardInterrupt:
```

```
    pass
```